

Project: Accelerating CNN Inference

Practical Course: Accelerating CNNs using PL

June 17, 2026

Contents

1	Introduction	2
2	Inference	3
2.1	Program	3
2.2	CNN::inference()	3
2.3	Project Folder	3
3	Hand-In, Report and Grading	4
3.1	General Guidelines	4
3.2	Grading Criteria	4
4	CNN Architecture	5
4.1	Linear Layer (Fully Connected Layer)	5
4.2	Activation Function	6
4.3	Max Pool	6
4.4	Average Pool	6
4.5	Convolutional Layer	6
5	Communicating with the IP	7
6	Tensor Class	8

1 Introduction

In the graded project you have to accelerate CNN Inference, using the FPGA on the board and compare it to the original CPU only implementation. The used network consists of 19 convolution layers and 1 Linear layer, as discussed in the lecture, and has an accuracy of 89% for the cifar-10 testset. The function `CNN::Inference(Tensor *, int N, uint8_t pred[])` performs inference on N input Tensors (Converted and normalized images) and stores the predicted output into the array `pred[]`. You are free to decide which parts (or all) of the inference function you want to move to the FPGA and what to keep on the CPU you should however explain your reasoning in the report. You can also change the precision or datatype of layers and operations, to reduce the resource utilization per operation. It is enough to do this empirically, e.g. test on the entire dataset whether the change in precision or datatype degrades the final precision significantly.

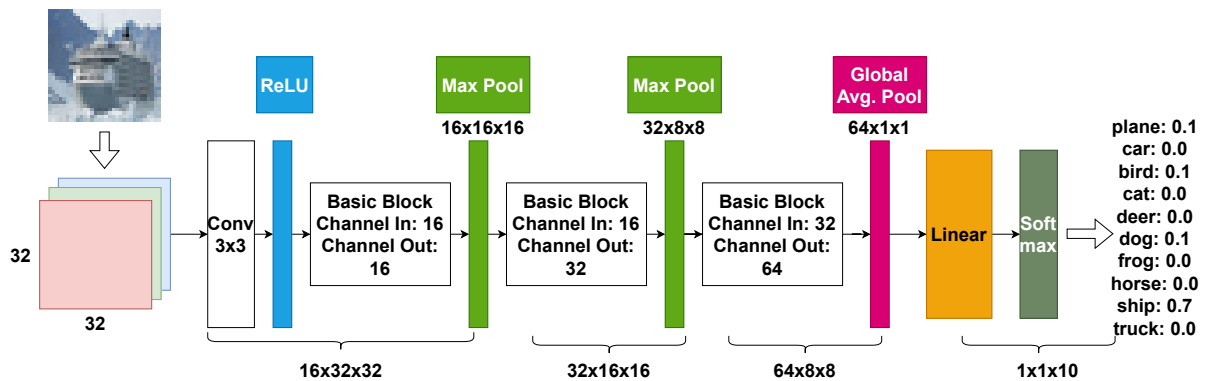


Figure 1 CNN layers

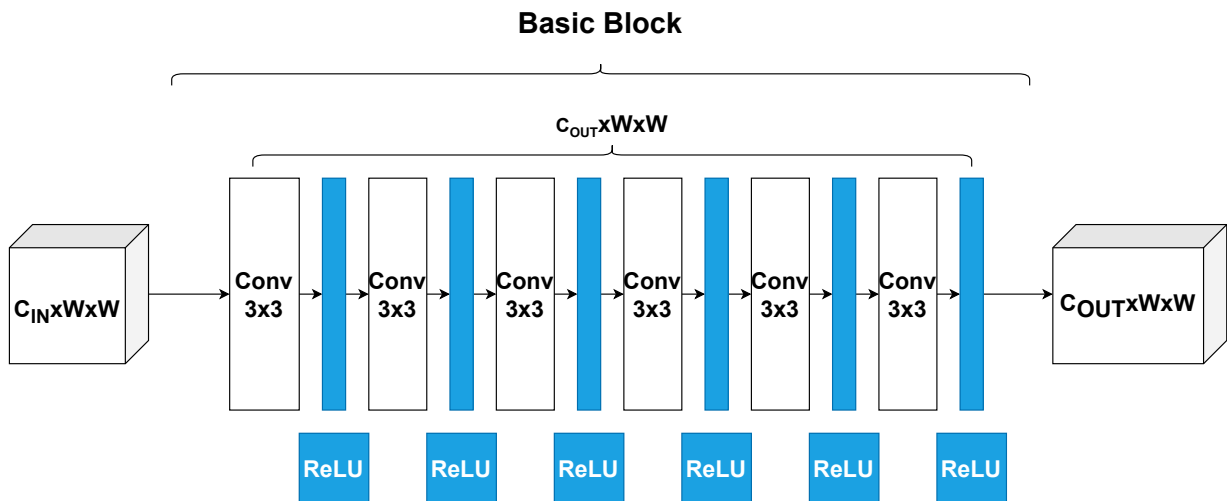


Figure 2 CNN basic blocks

2 Inference

2.1 Program

You can compile the source code using the provided makefile to test your implementation. The `inference.cpp` file contains three use cases, every use case reads the pre-trained parameters for the net from `data/fnet20_weights.dat`. **Make sure to change the project path inside the `inference.cpp` folder to your project folder.**

```
# Test:      ./inference t
# Predict:   ./inference p image.bmp
# Benchmark: ./inference b N
```

1. **Predict:** Performs inference on a single image of size 32×32 in a bmp format. The image is read and normalized before being passed to the inference function. You can find example images inside the images folder.
2. **Test:** Reads a test input (`./data/fnet20_test.dat`) and compares the output to a reference solution generated using Pytorch. For this function to work, the last tensor has to be stored in the last layers Z Tensor (`layers[42] -> Z`).
3. **Benchmark:** Reads the cifar-10 testset (`./data/cifar10_testset.dat`) and performs inference on every input image (max=10000) and returns an average accuracy and a timing report. You should use this function to measure your performance and you can use the input N to specify a maximum number of images.

2.2 CNN::inference()

This is the function you should accelerate, it loops through all inputs (N) and for each input iterates over all layers and passes the outputs of the previous to the next layer. It measures the time spent on each layer and outputs the average time after performing inference. The second part of the loop detects the maximum probability of the output vector and stores the index of the predicted class in the `pred[]` array. To use the parameters on the FPGA you will probably need to rearrange the data layout of the Tensor. All inputs and outputs of layers are 3D Tensors with all elements stored contiguously. The weights for each Conv layer are stored as a array of Tensors and the storage is not contiguous. You can convert all convolution weights into a single array using the `convWeightStorage()` function, and all Biases into a single array using the `convBiasStorage()` function (Both can be found inside the inference function and don't forget to delete the array after usage `delete []`). If you want to you can hard-code the layer dimensions inside your HLS implementation, the SW function is timed using the `mtick()`, `mtock()` functions from the previous lab (Not compiled under Windows).

2.3 Project Folder

The project folder contains a `src` directory including the file with the kernel implementations and the main `cnn` header and source file. The `util` directory includes utilities for constructing layers and reading the network weights. In addition it includes the files for the Tensor class and to read `.bmp` images and the CNN architecture definition. The `CNN::inference()` function already includes directives for compiling on the different architectures, these are only hints how you can separate the different versions and you are free to implement it differently. To compile the code you can either use the provided Makefile (change the HLS include folder) or import it to Vitis HLS and compile it there (you can have a look at the `hls.cfg` file on what to add). On the board you need to uncomment the board flags and comment the SW flags and it should compile without a problem. The `.xclbin` can be implemented as described in the previous lab and a later section describes the required xrt function calls in more detail.

3 Hand-In, Report and Grading

3.1 General Guidelines

The main hand-in of your project will be the final report, which will include a description of your implementation. It should include a detailed description of optimizations and how you integrated the FPGA with the code running on the ARM CPU. Explain how you have chosen which parts to move to Hardware, how you implemented them and how effective your implementation is. Support your report by including relevant performance benchmarks in the form of graphs and tables and a diagram of your HW architecture. Find bottlenecks in your implementation and try to explain their origin (e.g. Implementation, Memory, Compute Resources or Sequential Parts). Try to explain the result of your benchmark and if you use different datatypes than floats talk about the effect on accuracy (if any). Compare your optimized implementation with the original implementation running on the ARM chip. Finish your report with a conclusion and give an outlook for future optimizations.

The deadline for the Implementation is the 20.07 and the presentations will be on the 22.07, the report can be handed in until the 27.07 however it should be based on the handed in implementation apart from possible additional benchmarks.

The individual project discussions will be scheduled on a group-by-group basis.

The Report and Hand-In will be (50%) of your grade the report should be a .pdf file. The Code should be handed in an archive such that it can be compiled on the Pynq board with a single make command. The folder should include the .xclbin file as well as the original HLS source files.

3.2 Grading Criteria

Report & Implementation: (50%)

- Quality of Report (**Reasoning**, Benchmarks, Visualization,...)
- Quality of Implementation
 - What have you implemented (Complexity of Optimizations)
 - Deductions for major design flaws (Memory Leaks, extremely slow implementations)

Presentation (10%)

- 15 min + 5 min Questions
- Main focus: Hardware Part
- Quality (Slides, Presentation Style,...)

Individual Project Discussion (40%)

- Discussion about your handed-in code 15 min (Edge Detection + Project)
- Questions about implementation and choices for example:
 - You should understand **all** of the implementation (HLS, Port connections and Integration with the host code)
 - What do certain parts of the implementation do?
 - What do certain pragmas do, where are specific variables/arrays stored?
 - Why have you chosen a certain way over another?
- General Questions

4 CNN Architecture

The CNN consists of six different layer types that are described in later subsections. You can also find a C++ implementation of each layer inside the `kernels.cpp` file, for our network we only need 3×3 convolution. The network itself is defined inside the `nets.cpp` file. The network takes an input tensor of dimensions $3 \times 32 \times 32$ and outputs a probability distribution of 10 classes, all layers can be found in table 1. Every Tensor in the network before the linear layer has quadratic channels, meaning width and height are equivalent. All convolution layers are padded with zeros and use a kernel of size 3×3 and are followed by a ReLU layer that does not modify the Tensor dimensions.

Table 1 Plain resnet-20 layers (Input Tensor $3 \times 32 \times 32$)

Number	Type	Output Channels	Output Height×Width
1	Conv + ReLU	16	32x32
2	Conv + ReLU	16	32x32
3	Conv + ReLU	16	32x32
4	Conv + ReLU	16	32x32
5	Conv + ReLU	16	32x32
6	Conv + ReLU	16	32x32
7	Conv + ReLU	16	32x32
8	MaxPool	16	16x16
9	Conv + ReLU	32	16x16
10	Conv + ReLU	32	16x16
11	Conv + ReLU	32	16x16
12	Conv + ReLU	32	16x16
13	Conv + ReLU	32	16x16
14	Conv + ReLU	32	16x16
15	MaxPool	32	8x8
16	Conv + ReLU	64	8x8
17	Conv + ReLU	64	8x8
18	Conv + ReLU	64	8x8
19	Conv + ReLU	64	8x8
20	Conv + ReLU	64	8x8
21	Conv + ReLU	64	8x8
22	Avg Pool	64	1x1
23	Linear	1	1x10
24	Softmax	1	1x10

4.1 Linear Layer (Fully Connected Layer)

The Linear layer connects all inputs to all outputs in the form of a Perceptron:

$$\vec{z}(\vec{x}) = \mathbf{W}\vec{x} + b \quad (1)$$

Usually a Fully Connected Layer is followed by a nonlinear activation function, this can for example be a ReLU layer or a Softmax Layer if it is the final layer. If the previous layer is a multidimensional Tensor, the values have to be flattened into one dimension. This is done automatically as all data of a Tensor is stored sequentially (see later section).

4.2 Activation Function

The two activation functions used are $\text{ReLU}()$ and $\text{Softmax}()$ both work element-wise. The Softmax function return a probability between 0 and 1 for each classification element, and is used as the activation function of the last fully connected layer:

$$z_i(x_i) = \max(0, x_i) \quad (2)$$

$$z_i(x_i) = \frac{e^{x_i}}{\sum_{j=0}^{N-1} e^{x_j}} \quad (3)$$

4.3 Max Pool

Pooling performs a reduction in the input, by performing an operation on a field of the input for example averaging a 2×2 area of the input feature map or performing a max operation only passing along the maximum of the input values. Pooling Layers are usually not followed by an activation layer (Usually preceded by one). The Max Pool function in our example performs max pool with a size and stride of 2, thus halving the width of every input channel.

$$z[i, j] = \max(x[2 \times i, 2 \times j], x[2 \times i + 1, 2 \times j], x[2 \times i, 2 \times j + 1], x[2 \times i + 1, 2 \times j + 1]) \quad (4)$$

4.4 Average Pool

The average pool layer reduces every input channel to one value and is used to create a closer relationship between the independent features of the different input layers and the final linear layer. In our network there is one Average Pool layer reducing the input from $64 \times 8 \times 8$ to $64 \times 1 \times 1$.

$$z[i, 0, 0] = \frac{1}{W \times W} \sum_{q=0}^{W-1} \sum_{p=0}^{W-1} x[i, q, p] \quad (5)$$

4.5 Convolutional Layer

The convolutional Layer performs the convolution operation on an multidimensional input (Tensor). The key parameters of a convolutional layer are:

Input size The dimension of the Input tensor in 3D: Input Channels X_C , Feature Map height X_M and Feature Map width X_N .

Kernel (Weight) size The dimension of the Weight (Kernel) Tensor in 3D: Weight Channels W_C , Kernel height W_M and Kernel width W_N . The kernels in this practical course are all of size 3×3 use a stride of 1 for every convolutional layers. The number of channels per weight tensor is equal to the number of input channels $W_C = X_C$. For every output channel Z_C another Weight Tensor is required!

Bias Every Weight Tensor has one bias value attached that is added to every element of the output feature map. This means that the bias is a 1D-Tensor with one element for every output channel $B_N = Z_C$

Output size The output tensor only has one flexible parameter and that is the number of output channels Z_C , the width and height of the output is the same as the input channels as the convolution is padded with zeros.

Calculation of a single output

$$z_{i,j,k} = \sum_c \sum_p \sum_q^{X_C W_M W_N} x_{c,j+p,k+q} w_{i,c,p,q} \quad (6)$$

i indicates the output channel and j, k the y-and x-position of the output. Following a convolution layer is usually a non-linear activation function. The convolution layer can be padded to remain the dimensions, in the software implementation this is done by pre-padding the input.

5 Communicating with the IP

After implementing the IP in Vitis HLS and creating the .xclbin file you can allocate buffers and start the kernel as implemented in the previous lab. An example code snippet loading an example rnet20.xclbin file onto the board:

```
auto device = xrt::device(0);
auto xclbin_uuid = device.load_xclbin("./rnet20.xclbin");
loadBitstream("./rnet20.xclbin");
auto krnl = xrt::kernel(device, xclbin_uuid,
    "convEngine", xrt::kernel::cu_access_mode::exclusive);
```

Buffers accessed by m_axi interfaces have to be specially allocated using the buffer API:

```
auto xin = xrt::bo(device, sizeof(float) * 3*32*32, 0);
auto zout = xrt::bo(device, sizeof(float) * 64, 0);
auto wbuf = xrt::bo(device, sizeof(float) * w_total, 0);
auto bbuf = xrt::bo(device, sizeof(float) * b_total, 0);
```

Afterwards you can write to the buffers and read from the buffers using the write and read functions:

```
/* Write all Convolution weights and biases to a shared Buffer */
wbuf.write(w_all);
bbuf.write(b_all);
/* Write the input image tensor to shared memory */
xin.write(X->data[0][0]);
/* Read shared buffer z_out to the output Tensor with index 11*/
zout.read(layers[11].Z->data[0][0]);
```

There are two ways to start the kernel you can either start it and set all input arguments:

```
/* Start Kernel */
auto run = krnl(xin, wbuf, bbuf, zout);
/* Wait for kernel to finish */
run.wait();
```

or create the kernel and set the arguments and start it later:

```
/* Create Kernel */
xrt::run run(krnl);
/* Set 4 arguments */
run.set_arg(0, xin);
run.set_arg(1, wbuf);
run.set_arg(2, bbuf);
run.set_arg(3, zout);
/* Start and Wait for kernel */
run.start();
run.wait();
```

6 Tensor Class

The Tensor class is a 3d data structure that is used to store the Tensors between layers. A Tensor of size $Z \times Y \times X$ can be created using the constructor `Tensor(Z, Y, X)`. An element of a tensor can be accessed using brackets `e = T[z][y][x]` and all values are stored in one contiguous array so one could theoretically access the same element using `e = T[0][0][z*Y*X + y*X + x]`. To read a tensor from a file one can use the `read(FILE *f)` function, if the stored tensor is of different size than the Tensor calling the method the storage will be reallocated. To write a Tensor to a file the method `write(FILE * f)` can be used. Another useful function are the `randomize(FLOAT start, FLOAT stop)` that fills the Tensor with random values between start and stop.

Name:	Description
<code>T.size[0]</code>	Number of channels
<code>T.size[1]</code>	Tensor height
<code>T.size[2]</code>	Tensor width
<code>T[i]</code>	Returns 2d feature map
<code>T[i][j]</code>	Returns row of tensor
<code>T[i][j][k]</code>	Returns element of tensor
<code>Tensor()</code>	Creates empty Tensor with no memory allocation and size 0
<code>Tensor(i,j,k)</code>	Allocates Tensor of size: $i \times j \times k$
<code>read(f)</code>	Reads Tensor from file f (reallocates Tensor if required) returns status.
<code>write(f)</code>	Writes Tensor to file f
<code>resize(i,j,k)</code>	Resizes Tensor and reallocates data array
<code>randomize(start,stop)</code>	Fills Tensor with random values from start to stop

Table 2 Tensor Class Reference